

Ejercicio 9. Discusión

9.1 Características de DuckDB más útiles

La lectura de Parquet con comodines fue la base del trabajo. Una consulta sobre `data/raw/yellow/*/*.parquet` lee todos los meses sin cargarlos antes, y agregar 2024 y 2025 no cambió ninguna ruta.

La opción `union_by_name` resolvió los cambios de esquema entre años, como `cbd_congestion_fee` desde 2025 y `request_source` desde junio de 2026. Sin ella DuckDB descarta columnas o falla según el orden de los archivos.

Las funciones de metadatos, `parquet_file_metadata`, `parquet_schema` y `glob`, contaron 121 millones de registros, compararon esquemas y verificaron la descarga sin leer filas. `SUMMARIZE` perfiló las 21 columnas de 29.7 millones de viajes en segundos y mostró los valores imposibles de un vistazo.

El SQL analítico, con `FILTER`, `QUALIFY`, `ROLLUP`, `quantile_cont`, `UNION ALL BY NAME` y vistas, dejó cada pregunta en una consulta. DuckDB corre dentro de Python, así que la misma librería sirvió a los scripts y a los notebooks sin instalar un servidor, y Metabase lee con su driver de DuckDB la base de indicadores que arma scripts/tablero_metabase.py.

9.2 Ventajas y limitaciones de consultar los Parquet sin cargarlos

No hay paso de carga ni copia de los datos. Un archivo nuevo en `data/raw` entra en la siguiente consulta, DuckDB lee las columnas que necesita y usa las estadísticas del pie de cada archivo, y el formato lo leen otras herramientas. Con un mes de datos, la mediana de ninguna consulta del benchmark pasó de 0.43 segundos.

Cada consulta paga un costo fijo por abrir los archivos, leer sus pies y unir esquemas. Por eso, con 20 meses, un filtro de un día tardó 0.044 segundos sobre 40 archivos y 0.003 sobre una tabla. Los cambios de esquema obligan a usar `union_by_name` y a revisar nulos que no son errores. Con `cbd_congestion_fee` vacío en 2024, el porcentaje de viajes con cargo CBD de los dos años parecía de 71.8% cuando era de 30.1%. Los archivos no se pueden corregir con `UPDATE` o `DELETE`.

9.3 Ventajas y limitaciones de las tablas materializadas

Con un mes de datos la tabla fue más rápida en las siete consultas, de 1.2 a 7.6 veces. Con 20 meses mantuvo la ventaja en el filtro de un día (15 veces), en la unión con zonas (1.65 veces) y en la agregación por mes (1.3 veces). Guarda columnas ya calculadas, como el periodo, y acepta cambios con `UPDATE` y `DELETE`.

Crearla tomó 8.7 segundos con 72 millones de registros, ocupa 1.65 veces el espacio de los Parquet y queda desactualizada cuando llega un mes nuevo. En las dos consultas con función de ventana el optimizador eligió con la tabla un plan que recorre los datos dos veces, y la tabla fue entre 22% y 38% más lenta que los Parquet. Con un mes la tabla recupera su creación en dos rondas de las siete consultas. Con 20 meses las siete suman 9.65 segundos sobre los Parquet y 9.69 sobre la tabla, así que no la recupera.

9.4 Ventajas frente a cargar todo con pandas

Cargar con pandas enero de 2026 de taxis amarillos, 3.7 millones de filas que ocupan 64 MB en Parquet, dejó un DataFrame de 554 MB, unos 149 MB por millón de filas, y durante la lectura el proceso llegó a

Laboratorio 8. DuckDB

Javier Eduardo España (23361), Angel Esteban Esquit (23221), Roberto Jose Barreda (23354)

1.6 GB. A esa razón los 121 millones de registros de los tres años necesitarían cerca de 18 GB, más del doble de los 7.6 GB que Docker tiene en la computadora donde se ejecutó el laboratorio, antes de calcular nada.

DuckDB procesa los archivos por bloques y en paralelo, filtra y agrega antes de devolver datos a Python y puede usar disco cuando la memoria no alcanza. En este laboratorio pandas recibió tablas agregadas de decenas o cientos de filas para mostrarlas y graficarlas. Las consultas quedan escritas en SQL, separadas del código que dibuja las gráficas.

9.5 Características que permiten incorporar datos nuevos con cambios mínimos

Agregar 2024 cambió la lista ANIOS de scripts/download_data.py y la vista viajes, para que `cbd_congestion_fee` valga 0 en los meses sin esa columna. Agregar 2025 cambió solo esa lista. Las 44 consultas, los notebooks y el tablero de Metabase tomaron los tres años sin cambios.

Eso se debe a la lista ANIOS, que comparten la descarga y la verificación, a que la descarga omite lo que existe y reporta lo no publicado, a las rutas con comodín por tipo y año, a `union_by_name`, a las vistas de `sql/vistas.sql` como único punto de lectura y al periodo tomado del nombre de cada archivo.

9.6 Qué automatizar en un sistema de producción

La descarga mensual con su verificación contra la lista de la TLC, con aviso cuando falta un archivo o cambia su tamaño. Una revisión de esquema que avise cuando aparece o desaparece una columna, como `request_source` en junio de 2026. El indicador de registros excluidos con un umbral de alerta, que en enero de 2025 habría detectado las tarifas negativas del proveedor 2, cuando la exclusión de viajes amarillos pasó del 4.15% al 6.47%. La ejecución de todas las consultas de `sql` después de cada descarga, como en 5.7 y 8.3. Y la actualización del tablero. `scripts/tablero_metabase.py` calcula los indicadores una vez en `data/processed/tablero.duckdb` y Metabase lee esas tablas en lugar de los Parquet, así que basta con ejecutarlo después de cada descarga.

9.7 Decisiones de diseño para mantener el proyecto reproducible

El ambiente de Docker del repositorio base, con la imagen de Python, la versión de Metabase y las librerías de `requirements.txt` fijas. Los datos fuera de Git y de la imagen, descargados por script desde la fuente original y verificados contra la lista de la TLC. Cada consulta en un archivo `.sql` que los notebooks y los scripts ejecutan tal cual. La opción `--anios` del script de descarga, que permite repetir cada ejercicio con los años que usó. Rutas relativas a la raíz del repositorio y una semilla fija en las muestras. Notebooks que corren de principio a fin con `nbconvert` dentro del contenedor. El tablero de Metabase creado por script con su API, en lugar de armarlo a mano. Y las evidencias guardadas en docs, con los registros de descarga, las verificaciones y los tiempos del benchmark.

9.8 Lo que no habría sido evidente con datos pequeños

Los problemas raros aparecen con volumen, como fechas de 2001 en archivos de 2026, distancias de 328,522 millas o siete filas repetidas entre 30 millones. Los esquemas cambian con el tiempo, y con un archivo no se habría visto que `request_source` empieza en junio de 2026 ni que un promedio de varios años puede dejar fuera un año completo por sus nulos. La calidad cambia entre años. En 2025 el 17.63% de los viajes Flex Fare amarillos traía tarifa negativa, todos del proveedor 2.

El costo de una consulta depende de lo que calcula y del plan que elige el optimizador, más que de cuántas filas lee. Con 20 meses la tabla ganó 15 veces en un filtro de un día, empató en los cuartiles

Laboratorio 8. DuckDB

Javier Eduardo España (23361), Angel Esteban Esquit (23221), Roberto Jose Barreda (23354)

exactos y perdió en las consultas con ventana. La memoria se vuelve un límite. Con 2024 y 2026 juntos, la primera versión de la consulta de percentiles agotó los 7.6 GB del contenedor, y separarla en una agregación por variable lo resolvió. La primera ejecución de una consulta tarda más que las siguientes, hasta 2.4 veces en el filtro de un día sobre la tabla, así que un benchmark necesita repeticiones. Con pocos datos todo cabe en memoria y cualquier herramienta responde en un instante. Con 121 millones de registros importan la organización de los archivos, las columnas que se leen y el momento en que conviene materializar.